Environments, Packaging and Design Patterns

Module 3 — Onboarding & Industrialize the Codebase

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

After *Modularity and Python Modules*, `heron` is a clean, modular package, but it runs only with `PYTHONPATH=src`, and nothing says which Python or which tools it needs. Today we will configure the environment and make it installable.

Concepts covered: reproducible environments (virtual environments, dependency resolution, lockfiles, `uv`), `pyproject.toml`, extras and dependency groups, packaging and publishing (wheels, indexes), **semantic versioning** and conventional commits. Finally we will see some basic concepts about **design patterns**.

Reproducible environments

"Works on my machine"

Your teammate clones `heron`, runs it, and it crashes, while on your machine it works. The code is the same. The **environment** is not:

- a different Python version: you have 3.13, the server has 3.11
- different package versions: `pip install fastapi` today does not give the version it gave six months ago
- packages installed globally, so one project's packages leak into another
- something installed once, by hand, and forgotten

The industrial answer has three parts: **declare** what you need, **pin** the exact versions you got, **isolate** each project.

What "installing" a package means

`import requests` works only if Python finds a folder `requests/` in one of the places listed in `sys.path`. The main one is **`site-packages`**, a folder that belongs to one Python interpreter. **Installing** a package means putting it there:

- the installer (`pip`, `uv`) downloads a **distribution** of the package, usually a **wheel** (`.whl`): a zip file with the package's code and its **metadata** (name, version, dependencies, commands)
- it copies the code into `site-packages`, next to a `<name>-<version>.dist-info/` folder that records what was installed
- it installs every dependency in the same way
- it creates the package's **commands** (for example `pytest`), small scripts in the `bin/` folder next to the `python` executable

```
python3 -c "import sys; print(sys.path)"   # the folders where import looks
python3 -m pip show requests               # the installed version and its folder
```

`heron` was never installed: `PYTHONPATH=src` adds `src/` to `sys.path` by hand, for one command only. Today we make it installable.

Virtual environments

A **virtual environment** is a private Python installation for one project: a folder (usually `.venv/`) with a link to an interpreter and its own `site-packages`, where the project's packages are installed.

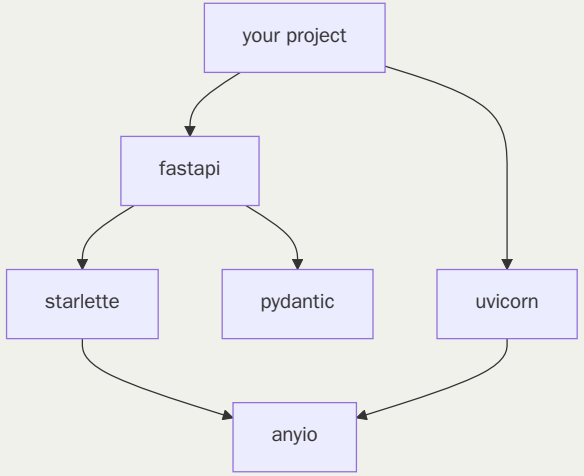
```
python3 -m venv .venv          # create it
source .venv/bin/activate      # use it in this shell
pip install -e .               # install the project into it ("editable")
```

An **editable** install (`-e`) links your source folder instead of copying it, so your edits take effect at once.

Rule: one environment per project, never the system Python. These three commands show what happens underneath; in this course `uv` (three slides ahead) does all of it for you.

Dependency resolution

You ask for two packages. Each of them needs other packages, which need others: a **graph** of dependencies, each with version constraints. A **resolver** finds one set of versions that satisfies every constraint at once.



`anyio` is needed through two paths, so the resolver must pick a version that both accept. When no version fits, you get a **conflict**. Doing this by hand, repeatably, is impossible: we use tools.

Lockfiles: declared vs resolved

Two files answer two different questions:

- **`pyproject.toml`**: what do we want? Names and ranges, such as `mypy` or `fastapi>=0.140`. Written by people.
- **`uv.lock`**: what did we get? Every package of the graph, with its exact version and a checksum (hash) of its file. Written by the tool.

What the lockfile gives you:

- **reproducibility**: your teammate, the CI server and production install exactly the same files
- **auditability**: a dependency change is a visible diff in a pull request
- the rule: an application **commits** its lockfile

uv

`uv` is a fast project manager for Python. It replaces several older tools at once: `pip` (install packages), `venv` (create environments), `pyenv` (install Python versions) and `pip-tools` (write lockfiles).

```
uv add requests               # declare a dependency, update the lock, install it
uv add --dev ruff             # a tool for developers only
uv sync                       # make .venv match the lockfile exactly
uv run heron data/samples     # run a command inside the project's environment
uv python install 3.12        # install a Python version
```

You never activate the environment by hand, and you never call `pip` directly: `uv run` always uses the declared, locked environment.

The tool family

`uv` is our choice, but the ideas are the same in every tool: declare, resolve, lock, run.

Tool	Lockfile	Notes
uv	`uv.lock`	fastest; also installs Python versions
poetry	`poetry.lock`	mature and popular; slower resolver
pip-tools	`requirements.txt` (compiled)	a thin layer over pip
pip + venv	none	the baseline; you pin versions by hand

Each tool writes its own lockfile format. **PEP 751** (accepted in 2025; a PEP, *Python Enhancement Proposal*, is the document that defines a Python standard) defines a common format, `pylock.toml`. `uv export --format pylock.toml` writes it from `uv.lock` when another tool needs it.

pyproject.toml

pyproject.toml: the project's identity card

`pyproject.toml`, at the root of the repository, is the **single** file that describes a Python project. It is written in TOML (a simple `key = value` format with `[sections]`). Its main tables:

- **`[project]`**: name, version, description, supported Python versions, licence, runtime dependencies
- **`[project.scripts]`**: the shell commands the package installs
- **`[build-system]`**: which tool builds the package
- **`[dependency-groups]`**: tools for the developers (tests, linters)
- **`[tool.<name>]`**: the configuration of each tool (ruff, mypy, pytest)

The next slides add these tables to `heron` one step at a time.

Step 1: declare heron

Create `pyproject.toml` at the root of the repository and type these lines. Then run `uv python pin 3.12`: it writes the file `.python-version`, which tells `uv` which Python to use.

```toml
[project]
name = "heron"
version = "0.2.0"
description = "Phishing email triage: parse .eml files, extract indicators, produce a verdict"
readme = "README.md"
requires-python = ">=3.12"
license = "MIT"  # SPDX expression (PEP 639)
dependencies = []  # core stays dependency-free; extras: [api], [ai]

[project.scripts]
heron = "heron.cli:main"

[build-system]
requires = ["hatchling>=1.27"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/heron"]
```

Step 1: what each line means

- `requires-python = ">=3.12"`: installers refuse older Pythons, which lack features `heron` uses
- `license = "MIT"`: an **SPDX** expression (SPDX is a standard list of licence names, used as defined by **PEP 639**)
- `dependencies = []`: `heron` needs only the standard library to run
- `[project.scripts]`: installing the package creates a shell command `heron` that calls the function `main()` of module `heron.cli`
- `[build-system]`: **hatchling** turns the source into an installable package; version 1.27 or newer understands SPDX licences
- `packages = ["src/heron"]`: where the package is (the src layout of *Modularity and Python Modules*)

```
uv run heron data/samples/prize-scam.eml   # uv creates .venv, installs heron in editable mode, runs it
# prize-scam.eml               phishing   score=9
git add pyproject.toml .python-version
```

No more `PYTHONPATH=src`: `heron` is now an **installed** package in its environment. `uv run` also writes a first `uv.lock`: leave it out of this commit, it is committed in step 2.

Extras and dependency groups

Optional packages go in two different places, for two different kinds of people:

	Extras `[project.optional-dependencies]`	**Dependency groups** `[dependency-groups]` **(PEP 735)**
For whom	**users** of the package	**developers** of the project
Published with the package?	yes	no, they never leave the repository
Install	`pip install "heron[api]"`, `uv sync --extra api`	`uv sync` (includes `dev`), `uv sync --no-dev`

Rule: if a **user** of the installed package could need it, it is an extra. If only **developers** need it (linters, test runners, documentation tools), it goes in a group. `heron` gets its first extras in Module 6 (`api`) and Module 8 (`ai`); today it gets the `dev` group.

Step 2: the dev tools and the lockfile

Nothing to type by hand: one command declares the developer tools, resolves every version, writes `uv.lock`, and installs everything into `.venv`.

```
uv add --dev ruff mypy pre-commit sphinx sphinx-rtd-theme
git add pyproject.toml uv.lock
```

It adds a `[dependency-groups]` table to `pyproject.toml`. `uv` writes each tool on its own line with a lower bound, such as `"ruff>=0.16.10"` (the version it just installed); keep what it writes.

For five tools, uv resolves 44 packages in the reference (including `heron` itself): open `uv.lock` and look. Your lockfile may list newer versions than the reference: it records what *you* got today, and that is its job. `uv.lock` is committed. A teammate then runs `uv sync --locked`: it rebuilds exactly the same `.venv`, or fails if the lockfile no longer matches `pyproject.toml`.

Packaging and publishing

Semantic versioning

Every release carries a **version number**. **Semantic versioning (semver)** gives it a meaning: **MAJOR.MINOR.PATCH**.

- **MAJOR** (1.4.2 → 2.0.0): a **breaking** change: code that worked with the old version may fail, for example a renamed command-line option or a changed return type
- **MINOR** (1.4.2 → 1.5.0): a new feature, **backward-compatible**: old usage still works
- **PATCH** (1.4.2 → 1.4.3): a **backward-compatible bug fix**

A user who reads only the version number knows whether an upgrade can break them. Versions before `1.0.0` (`0.y.z`) are for initial development: semver says that **anything may change at any time**, with any bump, so a user of a `0.y.z` package cannot rely on these promises.

Conventional commits and semver

Conventional commits is a format for commit messages that people and tools can read:

```
type(optional scope): short summary in the imperative

optional body: why the change was made
```

- types: `feat` (new feature), `fix` (bug fix), `refactor`, `docs`, `test`, `style` (layout only), `build` (packaging), `chore` (tooling, housekeeping), `ci`
- a breaking change adds `!` after the type (`feat!:`) or a `BREAKING CHANGE:` line in the body

The link to semver: from `1.0.0` on, a `feat` means a MINOR release, a `fix` a PATCH, a breaking change a MAJOR. Tools can read the history and choose the next version number.

While `heron` is at `0.y.z`, it follows a simpler rule of its own: the MINOR number goes up **once per module stage** (0.2.0 at the end of *Modularity and Python Modules*, 0.3.0 today), whatever the types of the commits.

Step 3: release 0.3.0

This module stage gives users something new and backward-compatible: `heron` is installable, with a `heron` command. So, by `heron`'s rule, the MINOR number goes up. The change:

```
uv version --bump minor       # heron 0.2.0 => 0.3.0 in pyproject.toml and uv.lock
```

Then edit `src/heron/__init__.py` by hand:

```
-__version__ = "0.2.0"
+__version__ = "0.3.0"
```

The version now lives in two places, `pyproject.toml` and `src/heron/__init__.py`, and you must keep them equal. (The alternative is `dynamic = ["version"]` in `[project]`, which tells hatchling to read the version from the code).

Build a distribution

With a build backend declared, `heron` can become a **distribution**: files that anyone can install. Try it after step 3:

```
uv build
# dist/heron-0.3.0-py3-none-any.whl
# dist/heron-0.3.0.tar.gz
unzip -l dist/heron-0.3.0-py3-none-any.whl
# heron/__init__.py  heron/cli.py  ...  heron-0.3.0.dist-info/METADATA  ...
```

- the **wheel** (`.whl`) is the built package of the slide "What installing a package means": the installer only copies its files and creates the `heron` command
- the **sdist** (`.tar.gz`, *source distribution*) holds the source; the installer must first build it on the target machine

`dist/` is in `.gitignore`: built files are outputs, not source, so `git status` stays clean.

Publish to an index

A **package index** is a server that hosts distributions: **PyPI** (pypi.org) is the public one; companies also run private indexes. After publishing, anyone can install the package by name:

```
uv publish                    # upload dist/* to the index (needs credentials)
pip install heron             # what a user then runs (or: uv add heron)
```

From a CI pipeline (Module 5), the safe way is **trusted publishing**: PyPI trusts a specific GitHub Actions workflow through a short-lived token signed by GitHub (**OIDC**, OpenID Connect), so no password or long-lived API token is stored anywhere.

We do not publish `heron` to PyPI: it is a course project, and the name belongs to someone else there. This is why you check a name before you publish.

Design patterns

What is a design pattern?

> *"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over." — Christopher Alexander, A Pattern Language (1977), about buildings*

Software engineering adopted the idea with the book *Design Patterns* by Gamma, Helm, Johnson and Vlissides (1994, the "Gang of Four"), which describes 23 patterns.

The core idea of most patterns: **isolate what changes**. Put the parts that are likely to change behind a stable structure, so that a change does not spread through the code.

The four elements of a pattern

1. a **name**: one or two words that the whole team understands ("use a Strategy here")
2. the **problem**: when to apply the pattern, and in which context
3. the **solution**: the elements of the design, their responsibilities and how they collaborate. It is a template, not a finished piece of code: each application looks different
4. the **consequences**: the results and the trade-offs of applying it

Patterns are grouped by purpose: **creational** (how objects are created), **structural** (how objects and modules are connected), **behavioural** (how responsibilities and algorithms are divided). At the lowest level, **idioms** are the patterns of one language (for example `with open(...)` in Python).

Patterns in heron

Many classic patterns, designed for languages such as Java and C++, become much simpler in Python, because functions are objects and a module is already a single shared instance.

`heron` really uses three patterns, in the code you wrote in *Modularity and Python Modules* (this lecture and *Documentation and Quality Gates* do not change the design):

Pattern	Kind	Where in `heron`
Strategy-like	behavioural	the checks in `ALL_CHECKS` (`features.py`)
Facade	structural	`triage()` in `scorer.py`
Adapter	structural	`parse_eml()` in `parser.py`

Facade, in one sentence: one simple entry point in front of a subsystem. `triage()` (step 7 of *Modularity and Python Modules*) hides the five checks, the weights and the thresholds behind one call that returns a `TriageResult`.

Strategy, and heron's checks

Problem: a task can be done with several interchangeable algorithms, and you want to choose or swap them without changing the code that uses them.

Solution: give every algorithm the same interface; the client holds **one** of them and calls it. In Python a strategy is simply a function: in `sorted(emails, key=...)`, the `key` function is the strategy that decides the order.

In `heron`: every check has the same signature (it takes a `ParsedEmail` and returns its points as an `int`), and `triage()` loops over `ALL_CHECKS` (steps 6 and 7 of *Modularity and Python Modules*). The difference: a Strategy **selects one** algorithm, while `triage()` **runs all** the checks and adds up their points. So we call it **Strategy-like**: pluggable checks with one interface.

Consequence: the scorer never names a check. A new indicator is one new function plus one line in `ALL_CHECKS` (the open–closed principle of *Modularity and Python Modules*).

Adapter

Problem: you must use a component whose interface does not fit your code, and you do not own it.

Solution: a small piece of code converts one interface into the other. `parse_eml()` adapts the standard library's email message (headers, MIME parts, encodings) to `heron`'s own `ParsedEmail`:

```python
def parse_eml(raw: str, email_id: str) -> ParsedEmail:
    """Read one raw message once and keep only what the checks need."""
    msg = message_from_string(raw, policy=policy.default)
    sender = str(msg.get("From", ""))
    return ParsedEmail(
        ...  # the seven fields, read from msg (step 4 of *Modularity and Python Modules*)
    )
```

Consequence: this is the isolation layer of *Code Quality and Refactoring*. If we change email library, only `parser.py` changes. Together with the facade `triage()`, it is all that `cli.py` needs from the domain: `triage(parse_eml(raw, file.name))`.

Python decorators are not the Decorator pattern

The names are the same, the ideas are not:

- the **Decorator pattern** wraps an **object** in another object with the **same interface**, which adds behaviour before or after each call. The client cannot tell the difference
- a **Python decorator** is the syntax `@name` above a `def` or `class`: Python passes the function or class to `name` and keeps what it returns. It often wraps a function, but not always

`@dataclass` in `models.py` is a Python decorator that wraps **nothing**: it adds methods (`__init__`, `__repr__`, `__eq__`) to the class and returns the same class.

```python
@dataclass(frozen=True)
class TriageResult:
    """The verdict on one message, with its score."""

    email_id: str
    score: int
    verdict: Verdict
```

`heron` gets a real Decorator in Module 8: a tracing client that wraps the LLM client, has the same methods, and records every call.

Patterns: a warning

Patterns solve problems you **actually have**. Applied "just in case", they become **over-engineering**: extra classes and layers that make simple code hard to read.

- a Factory (an object that decides which class to create) is worth it when building the object is complex or the class must be chosen at run time; `Foo()` needs no `FooFactory`. `build_parser()` in `cli.py` always builds the same object: it is a helper function, not a Factory
- a Strategy is worth it when there are really several algorithms; one `if` is not a Strategy
- AI coding tools often over-engineer: "remove this abstraction" will be a review comment you write often (Module 7)

`heron` gets more patterns only when a module needs them: you will meet them in Modules 6 and 8, in the code that uses them.

Where we are

Look at your history and run the installed command:

```
git log --oneline -3                    # your three commits of today
uv run heron data/samples/prize-scam.eml
# prize-scam.eml               phishing   score=9
```

`heron` is now declared, installable and locked, at version 0.3.0. What is still missing:

- nothing **checks** layout, likely bugs or types automatically, and there is no documentation site: *Documentation and Quality Gates* adds them, on the same branch
- nothing **proves** it works: no tests (Module 4)

Takeaways

- "works on my machine" is fixed by **declaring** dependencies (`pyproject.toml`), **pinning** them (`uv.lock`, committed) and **isolating** each project (`.venv`)
- `uv` manages environments, dependencies, the lockfile and Python versions; `uv sync --locked` rebuilds the same environment anywhere
- `pyproject.toml` declares the project; extras are for users, dependency groups for developers
- `uv build` makes a wheel and an sdist; an index (PyPI) distributes them; **semver** (MAJOR.MINOR.PATCH) tells users what an upgrade means from `1.0.0` on, and conventional commits map onto it; at `0.y.z` anything may change
- a **design pattern** is a named solution to a recurring problem; `heron` uses Strategy-like checks (`ALL_CHECKS`), a Facade (`triage()`) and an Adapter (`parse_eml()`); `@dataclass` is a Python decorator, not the Decorator pattern; add a pattern only when you have its problem

Appendix: every step in full

How to read the appendix

The slides of the lecture show the key lines of each step. The next slides show the **complete change** of every step, as `git diff` prints it, so that you can check your files line by line, or type a step you missed.

- a line that starts with `-` is removed, a line that starts with `+` is added, a line that starts with a space is unchanged and only shows where the change goes
- `@@ -12,2 +12,7 @@` gives the position of the lines that follow: line 12 of the old file (2 lines shown) becomes line 12 of the new file (7 lines); `-0,0` means a new file
- files that a tool writes (`uv.lock`) are not shown: the slide says which command writes them
- a new file (`pyproject.toml`) is written from scratch

Step 1 in full (1/2)

Commit `build: declare the project in pyproject.toml`: the complete change. `pyproject.toml` is a new file, written from scratch:

```
@@ -0,0 +1,18 @@
+[project]
+name = "heron"
+version = "0.2.0"
+description = "Phishing email triage: parse .eml files, extract indicators, produce a verdict"
+readme = "README.md"
+requires-python = ">=3.12"
+license = "MIT"  # SPDX expression (PEP 639)
+dependencies = []  # core stays dependency-free; extras: [api], [ai]
+
+[project.scripts]
+heron = "heron.cli:main"
+
+[build-system]
+requires = ["hatchling>=1.27"]
+build-backend = "hatchling.build"
+
+[tool.hatch.build.targets.wheel]
+packages = ["src/heron"]
```

Step 1 in full (2/2)

Step 1, continued: `uv python pin 3.12` writes the new file `.python-version`, one line long. Add both files before the commit.

```
@@ -0,0 +1 @@
+3.12
```

Step 2 in full

Commit `chore: add the dev tools with uv and commit uv.lock`: the change to `pyproject.toml`. `uv add --dev …` writes the table with one tool per line and a version bound; my commit lists the names only, with a comment added by hand. The new `uv.lock` (about 900 lines) is written by `uv`: never edit it by hand.

```
@@ -12,2 +12,7 @@ heron = "heron.cli:main"

+[dependency-groups]  # developer tools: never published; `uv sync` installs them
+dev = [
+  "ruff", "mypy", "pre-commit", "sphinx", "sphinx-rtd-theme",
+]
+
 [build-system]
```

Step 3 in full

Commit `chore: release 0.3.0`: `uv version --bump minor` changes `pyproject.toml` (and the `heron` entry in `uv.lock`); `src/heron/__init__.py` is edited by hand. Add the three files. First `pyproject.toml`:

```
@@ -2,3 +2,3 @@
 name = "heron"
-version = "0.2.0"
+version = "0.3.0"
 description = "Phishing email triage: parse .eml files, extract indicators, produce a verdict"
```

Then `src/heron/__init__.py`:

```
@@ -2,2 +2,2 @@

-__version__ = "0.2.0"
+__version__ = "0.3.0"
```